

Proposal for Orbital 26

Team Name:

0x02

Proposed Level of Achievement:

Apollo 11

Motivation

Modern note-taking apps often prioritize a "click-heavy" visual interface that disrupts the "flow state" of power users. For developers and students, the constant context-switching between the keyboard and mouse is an ergonomic bottleneck that slows down thought-to-text translation.

Aim

To build a keyboard-first note-taking editor where structure, computation, and AI assistance can be triggered without leaving the typing flow. By utilizing a "Command-Line Interface (CLI) within a Doc" approach, we hope to provide a seamless experience where structural changes, code execution, and AI assistance are all triggered via the home row. The best part of this would be that it will all be done without the need to memorise any keyboard shortcuts!

User Stories

1. **The Focused Student:** As a student taking fast-paced lecture notes, I want to create complex tables using only `Tab` and `Enter` so that I can structure information without interrupting my typing flow.
2. **The Agile Developer:** As a coder brainstorming logic, I want to type `//code` to run a snippet and see the output in my notes to verify my ideas immediately.
3. **The Academic Writer:** As an essay writer, I want to type `\\prompt` to get instant AI feedback or expansion without leaving my editor.

Core Features & Development Timeline:

Feature 1 (Core): The Notepad (18 May - 31 May)

- Blank page for users to type
- Has all standard features with good markup system
- Able to save and export (.x2pad format)

Feature 2 (Core): The `//` Registry (1 Jun - 11 Jun)

- Type an action without having to click the mouse
- Change font sizes and text formats, create tables and code boxes, etc.
- `"//<cmd> enter"`

Feature 3 (Core): The `\\` Registry (12 Jun - 29 Jun)

- AI search
- Generate suggestions or answer questions user has
- `"\\<prompt> enter"`

Feature 4 (Extension): Code Box (1 Jul - 10 Jul)

- Allow users to type code with standard code formatting
- Run C++ and Python code (may extend to more languages if time permits)

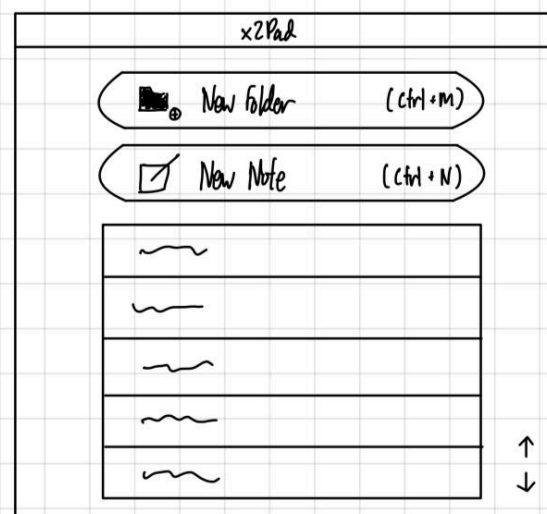
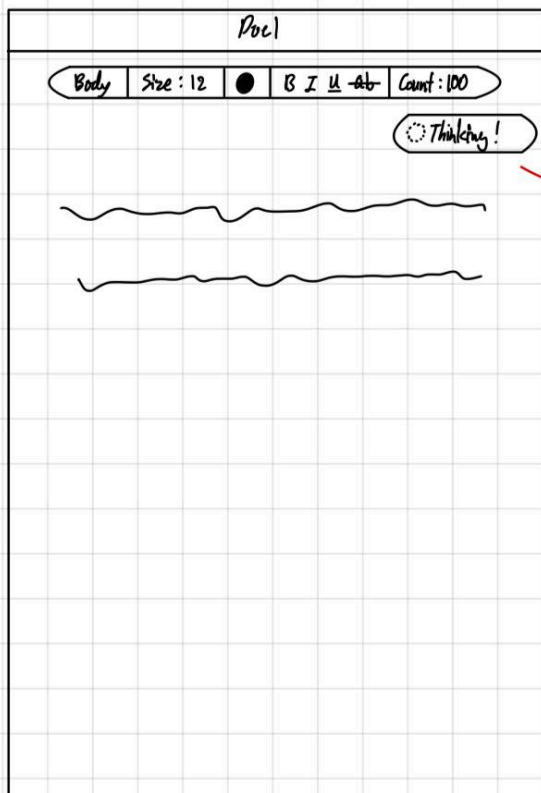
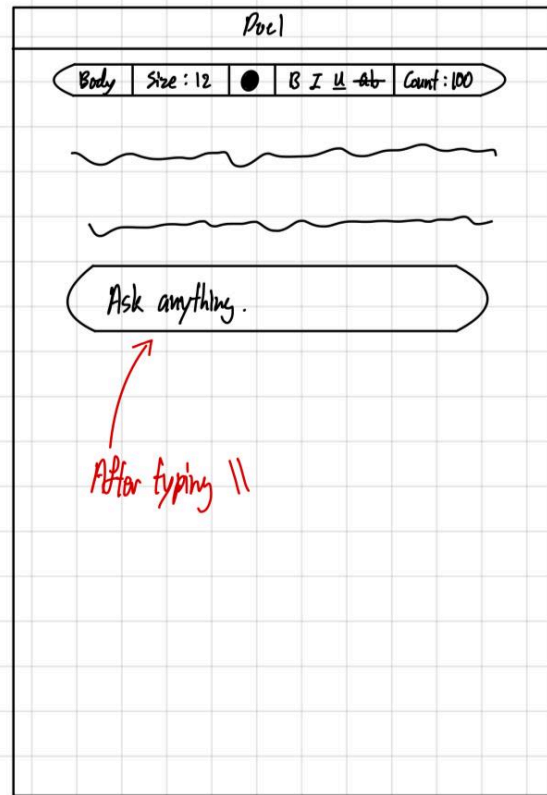
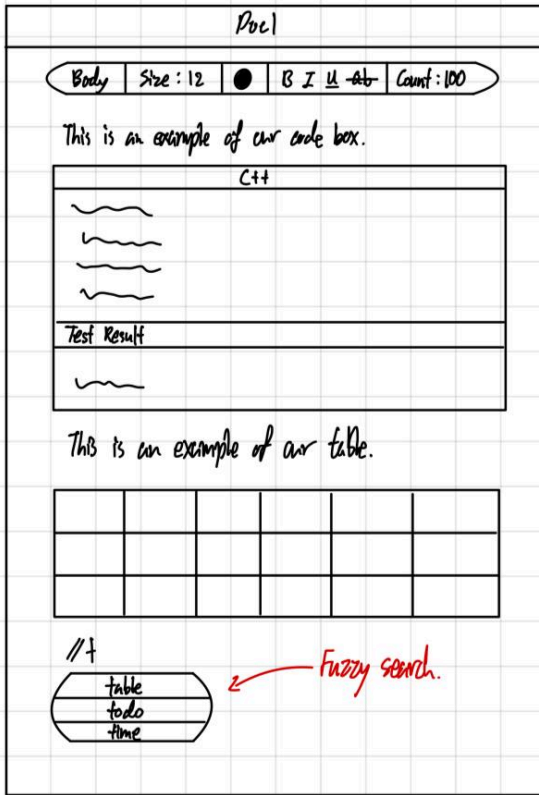
Feature 5 (Extension): Tables (11 Jul - 20 Jul)

- *Enter* and *tab* to extend rows and columns
- Table commands such as SUM, AVERAGE, etc.

Feature 6 (Extension): Fuzzy Search (21 Jul - 27 Jul)

- Gives suggestions when users type "/"
- Given in dropbox format, and users can select using up/down arrows
- Eliminates need to memorise commands

Ideas



Commands Registry:	Excel Commands:
1. //title, //header, //body 2. //bold, //italic, //strike, //underline 3. //font 4. //color 5. //bulletlist, //numberlist 6. //linebreak 7. //code 8. //table 9. //date, //time 10. //wordcount 11. //save, //export	1. //sum() 2. //avg(), //mean(), //median() 3. //min(), //max() 4. //count()

Testing & Quality Assurance Ideas

1. **Sandboxing** - Isolated testing environment to ensure working features
2. **Cross-Format Integration Testing** - Ensuring that all features are able to work together without conflicts.
3. **Stress Testing** - Loading large files (10000+ lines) into the app to check for responsiveness
4. **Edge-case Input Testing** - Inputting non-standard text inputs, checking that they do not run as code

Tech Stack

1. Tauri V2
2. CodeMirror 6
3. Rust
4. OpenAI API
5. Fuse.js

Qualifications

1. Docker implementations
2. HTML
3. UI/UX design
4. Data structure and algorithms
5. Discrete structures
6. Programming Methodology

Software Engineering

- **Documentation**-Driven Development
 - Clear project log
- Robust **API** & Error Handling (AI Integration)
 - Graceful Degradation and Asynchronous Programming.
- Inputting **Interceptors**

- Implementing a **CommandRegistry** (for all commands)
- **CI/CD**
 - Use Github actions to automatically build and test app for windows/mac on every push
- **State** management
 - Ensure document state remains consistent and no data is lost
- **Sandboxing**
 - Explore docker to sandbox the “code execution” feature so it cannot accidentally harm the user’s host system
- **Asynchronous processing**
 - AI’s responses will be handled asynchronously so the UI never freezes while waiting for AI to respond